

## LAB 01 | المختبر 01

# Neural Network Training

تدريب شبكة عصبية من الحل الابتدائي حتى التقارب

Forward Propagation, Backpropagation, Weight Update, and Convergence

الانتشار الأمامي والانتشار الخلفي وتحديث الأوزان والوصول إلى التقارب

Prepared by | إعداد: Dr.-Ing. Aouss Gabash

## 🎯 Lab Objectives

- Implement a small neural network in MATLAB.
- Calculate forward propagation step by step.
- Calculate the loss function.
- Implement backpropagation using analytical gradients.
- Update weights using gradient descent.
- Stop training when the error becomes smaller than  $\epsilon$ .

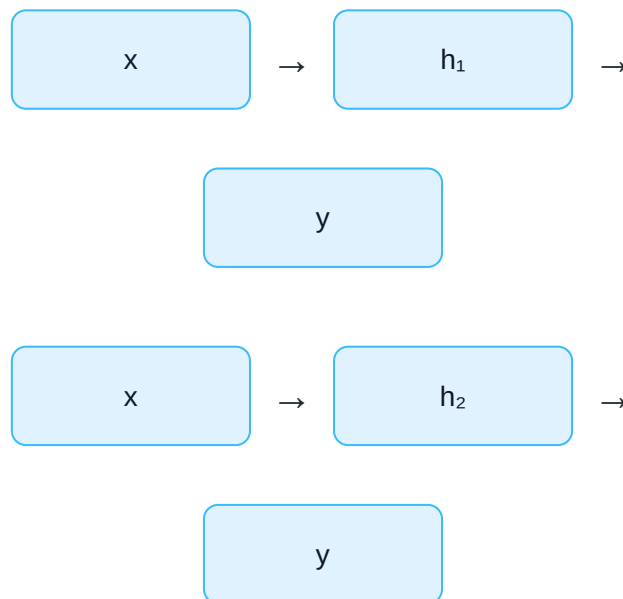
## 🎯 أهداف المختبر

- تنفيذ شبكة عصبية صغيرة باستخدام MATLAB.
- حساب الانتشار الأمامي خطوة بخطوة.
- حساب دالة الخطأ.
- تنفيذ الانتشار الخلفي باستخدام المشتقات التحليلية.
- تحديث الأوزان باستخدام الانحدار المتدرج.
- إيقاف التدريب عندما يصبح الخطأ أصغر من  $\epsilon$ .

## 1. Network Structure

We use a simple 1–2–1 neural network:

1 Input → 2 Hidden Neurons → 1 Output



## 1. بنية الشبكة

نستخدم شبكة بسيطة من النوع:

$1 \rightarrow 2 \rightarrow 1$

أي دخل واحد، عصبونان في الطبقة المخفية، وخرج واحد.

## 2. Activation Function

We use the Sigmoid activation function:

$$\sigma(z) = 1/(1 + e^{-z})$$

Its derivative is:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

## 2. دالة التنشيط

نستخدم دالة Sigmoid:

$$\sigma(z) = 1/(1 + e^{-z})$$

ومشتقتها:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

## 3. MATLAB Sigmoid Function

```
sigma = @(z) 1 ./ (1 + exp(-z));
```

This anonymous function calculates the Sigmoid value for any input z.

## 3. دالة Sigmoid في MATLAB

```
sigma = @(z) 1 ./ (1 + exp(-z));
```

يحسب هذا السطر قيمة Sigmoid لأي قيمة z.

## 4. Initial Parameters

Start with initial weights and biases:

```
W.w1 = 0.4;  
W.w2 = -0.3;  
  
W.v1 = 0.2;  
W.v2 = 0.5;  
  
W.b1 = 0.1;  
W.b2 = -0.1;  
W.b3 = 0.0;
```

These are only initial values. Training will modify them.

## 4. القيم الابتدائية

نبدأ بأوزان وانزياحات ابتدائية.

هذه ليست القيم النهائية؛ ستقوم عملية التدريب بتعديلها تدريجيًا.

## 5. Training Sample

```
x = 1;  
t = 0.8;
```

Here:

$x = input$

$t = desiredtarget$

## 5. عينة التدريب

$x = 1;$   
 $t = 0.8;$

حيث  $x$  هو الدخل، و  $t$  هي القيمة المطلوبة من الشبكة.

## 6. Forward Propagation

Hidden neuron 1:

$$z_1 = w_1x + b_1$$

$$h_1 = \sigma(z_1)$$

Hidden neuron 2:

$$z_2 = w_2x + b_2$$

$$h_2 = \sigma(z_2)$$

Output neuron:

$$z_3 = v_1h_1 + v_2h_2 + b_3$$

$$y = \sigma(z_3)$$

## 6. الانتشار الأمامي

يتم حساب خرج كل عصبون في الطبقة المخفية، ثم استخدام هذه القيم لحساب خرج الشبكة النهائي.

## 7. MATLAB Forward Propagation

```
z1 = W.w1*x + W.b1;
h1 = sigma(z1);

z2 = W.w2*x + W.b2;
h2 = sigma(z2);

z3 = W.v1*h1 + W.v2*h2 + W.b3;
y = sigma(z3);
```

## 7. الانتشار الأمامي في MATLAB

هذه الأسطر تنفذ المعادلات الرياضية نفسها مباشرة في MATLAB.

## 8. Loss Function

We use:

$$E = 1/2(y - t)^2$$

MATLAB:

```
E = 0.5*(y - t)^2;
```

## 8. دالة الخطأ

$$E = 1/2(y - t)^2$$

كلما اقترب  $y$  من  $t$  انخفضت قيمة  $E$ .

## 9. Backpropagation: Output Error

From Lecture 05:

$$\delta_3 = (y - t)y(1 - y)$$

MATLAB:

```
d0 = (y - t) * y * (1 - y);
```

dO is the derivative  $\partial E / \partial z_3$ .

9. الانتشار الخلفي: خطأ طبقة الخرج

$$\delta_3 = (y - t)y(1 - y)$$

تمثل dO تأثير دخل عصبون الخرج  $z_3$  على الخطأ النهائي.

## 10. Output-Layer Gradients

$$\partial E / \partial v_1 = \delta_3 h_1$$

$$\partial E / \partial v_2 = \delta_3 h_2$$

$$\partial E / \partial b_3 = \delta_3$$

```
dV1 = d0*h1;
dV2 = d0*h2;
dB3 = d0;
```

### 10. تدرجات طبقة الخرج

نحسب تأثير كل وزن وانزياح في طبقة الخرج على دالة الخطأ.

## 11. Hidden-Layer Error

$$\delta_1 = \delta_3 v_1 h_1 (1 - h_1)$$

$$\delta_2 = \delta_3 v_2 h_2 (1 - h_2)$$

```
d1 = d0*W.v1*h1*(1-h1);
d2 = d0*W.v2*h2*(1-h2);
```



## 11. خطأ الطبقة المخفية

يتم إرجاع تأثير الخطأ من طبقة الخرج إلى العصبونات المخفية عبر الأوزان  $v_1$  و  $v_2$ .

## 12. Input-to-Hidden Gradients

$$\partial E / \partial w_1 = \delta_1 x$$

$$\partial E / \partial w_2 = \delta_2 x$$

$$dW1 = d1 * x;$$

$$dB1 = d1;$$

$$dW2 = d2 * x;$$

$$dB2 = d2;$$

## 12. تدرجات الأوزان بين الدخل والطبقة المخفية

نحصل أخيرًا على مشتقات الخطأ بالنسبة للأوزان والانزياحات الأولى.

## 13. Gradient Descent Update

$$w(new) = w(old) - \eta \partial E / \partial w$$

```

W.w1 = W.w1 - eta*dW1;
W.w2 = W.w2 - eta*dW2;

W.v1 = W.v1 - eta*dV1;
W.v2 = W.v2 - eta*dV2;

W.b1 = W.b1 - eta*dB1;
W.b2 = W.b2 - eta*dB2;
W.b3 = W.b3 - eta*dB3;

```

## 13. تحديث الأوزان

نحدث كل معامل بعكس اتجاه التدرج حتى ينخفض الخطأ.

$$w(new) = w(old) - \eta \partial E / \partial w$$

## 14. Learning Rate

Choose:

eta = 0.5;

**$\eta$  too small**

Training becomes slow.

**$\eta$  too large**

Training may oscillate or diverge.

## 14. معدل التعلم

إذا كان معدل التعلم صغيرًا جدًا يصبح التدريب بطيئًا، وإذا كان كبيرًا جدًا فقد يصبح التدريب غير مستقر.

## 15. Stopping Criterion

Define a required accuracy:

```
epsilon = 1e-6;
```

Stop when:

$$E < \varepsilon$$

and also use a maximum epoch limit:

```
maxEpochs = 10000;
```

## 15. معيار التوقف

يتوقف التدريب عندما يصبح الخطأ أصغر من الدقة المطلوبة  $\epsilon$ ، مع وضع حد أقصى لعدد دورات التدريب لتجنب حلقة لا نهائية.

## 16. Complete MATLAB Program

```
clear; clc; close all;

sigma = @(z) 1 ./ (1 + exp(-z));

x = 1;
t = 0.8;

eta = 0.5;
epsilon = 1e-6;
maxEpochs = 10000;

W.w1 = 0.4;
W.w2 = -0.3;

W.v1 = 0.2;
W.v2 = 0.5;

W.b1 = 0.1;
W.b2 = -0.1;
W.b3 = 0.0;

lossHist = zeros(1,maxEpochs);

epoch = 0;
converged = false;

while ~converged && epoch < maxEpochs

    epoch = epoch + 1;

    % ----- Forward propagation -----

    z1 = W.w1*x + W.b1;
    h1 = sigma(z1);

    z2 = W.w2*x + W.b2;
    h2 = sigma(z2);

    z3 = W.v1*h1 + W.v2*h2 + W.b3;
    y = sigma(z3);

    % ----- Loss -----

    E = 0.5*(y - t)^2;
```

```

    lossHist(epoch) = E;

    % ----- Backpropagation -----

    d0 = (y - t) * y * (1 - y);

    dV1 = d0*h1;
    dV2 = d0*h2;
    dB3 = d0;

    d1 = d0*W.v1*h1*(1-h1);
    d2 = d0*W.v2*h2*(1-h2);

    dW1 = d1*x;
    dB1 = d1;

    dW2 = d2*x;
    dB2 = d2;

    % ----- Update parameters -----

    W.w1 = W.w1 - eta*dW1;
    W.w2 = W.w2 - eta*dW2;

    W.v1 = W.v1 - eta*dV1;
    W.v2 = W.v2 - eta*dV2;

    W.b1 = W.b1 - eta*dB1;
    W.b2 = W.b2 - eta*dB2;
    W.b3 = W.b3 - eta*dB3;

    % ----- Stopping criterion -----

    if E < epsilon
        converged = true;
    end
end

disp('Final weights and biases:')
disp(W)

fprintf('Epochs = %d\n',epoch)
fprintf('Final output y = %.6f\n',y)
fprintf('Target t = %.6f\n',t)
fprintf('Final error E = %.10f\n',E)

```

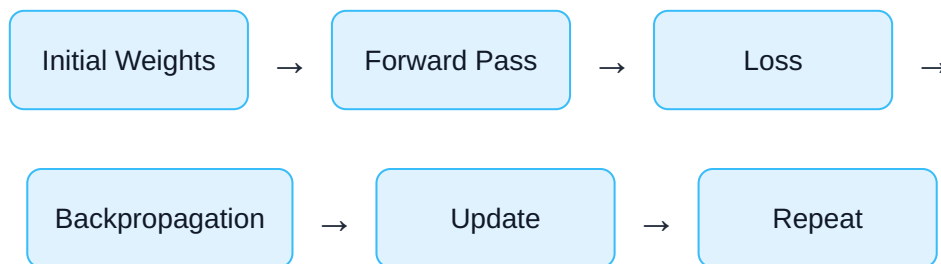
```
figure
semilogy(1:epoch,lossHist(1:epoch),'LineWidth',1.6)
grid on

xlabel('Epoch')
ylabel('Loss E')
title('Neural Network Training Convergence')
```

## 16. البرنامج الكامل في MATLAB

يجمع هذا البرنامج جميع الخطوات السابقة في حلقة تدريب واحدة: الانتشار الأمامي، حساب الخطأ، الانتشار الخلفي، تحديث الأوزان، ثم فحص شرط التقارب.

## 17. Understanding the Training Loop



The network learns because every iteration slightly modifies the parameters in a direction that reduces the error.

## 17. فهم حلقة التدريب

الأوزان الابتدائية ← الانتشار الأمامي ← الخطأ ← الانتشار الخلفي ← تحديث الأوزان ← التكرار حتى التقارب

## 18. Convergence Plot

The command:

```
semilogy(1:epoch,lossHist(1:epoch),'LineWidth',1.6)
```



plots the loss on a logarithmic vertical axis.

A descending curve indicates that the network is learning.

## 18. منحنى التقارب

يستخدم الأمر semilogy مقياسًا لوغاريتميًا لمحور الخطأ، وهذا يجعل متابعة انخفاض الخطأ عبر عدة مراتب عشرية أكثر وضوحًا.

## 19. Experiment 1 — Change Learning Rate

Try:

```
eta = 0.05;
eta = 0.5;
eta = 2.0;
```

Compare:

- Number of epochs
- Stability
- Final loss

## 19. التجربة 1 — تغيير معدل التعلم

غير قيمة  $\eta$  وقارن سرعة التقارب واستقرار عملية التدريب.

## 20. Experiment 2 — Change Initial Weights

Modify the initial values of:

```
W.w1
W.w2
W.v1
W.v2
```

Different initial conditions may produce different training paths.

## 20. التجربة 2 — تغيير الأوزان الابتدائية

غير القيم الابتدائية ولاحظ تأثيرها على مسار التدريب وعدد الدورات المطلوبة للوصول إلى التقارب.

## 21. Power-System Interpretation

The same training mechanism can be used for a power-system prediction problem.

For example:

$$x = PreviousLoad$$

$$t = FutureLoad$$

With more inputs:

$$x = [Load, Temperature, Time, PVPower, ...]$$

The mathematics of backpropagation remains the same.

## 21. التفسير في نظم الطاقة

يمكن استخدام آلية التدريب نفسها للتنبؤ بالحمل أو الطاقة المتجددة أو تقدير حالات التشغيل.

يتغير معنى المدخلات والمخرجات، لكن مبدأ الانتشار الخلفي يبقى نفسه.

## 22. Lab Tasks

1. Run the original program.
2. Record the final output and number of epochs.
3. Change the learning rate and compare convergence.

4. Change the initial weights.
5. Change the target value  $t$ .
6. Explain every gradient expression mathematically.
7. Explain why the loss decreases during training.

## 22. مهام المختبر

1. شغل البرنامج الأصلي.
2. سجل الخرج النهائي وعدد دورات التدريب.
3. غيّر معدل التعلم وقارن التقارب.
4. غيّر الأوزان الابتدائية.
5. غيّر القيمة المطلوبة  $t$ .
6. اشرح رياضياً كل معادلة من معادلات التدرج.
7. اشرح لماذا ينخفض الخطأ أثناء التدريب.



## Questions

1. What does  $dO$  represent?
2. Why does  $dO$  contain  $y(1-y)$ ?
3. Why is  $dV1$  equal to  $dO * h1$ ?
4. Why does the hidden error contain  $W.v1$ ?
5. Why do we subtract the gradient?
6. What happens if  $\eta$  is too large?
7. What is the role of  $\epsilon$ ?

## أسئلة المختبر

1. ماذا تمثل  $dO$ ؟
2. لماذا تحتوي  $dO$  على  $y(1-y)$ ؟
3. لماذا تساوي  $dV1$  القيمة  $dO * h1$ ؟
4. لماذا يحتوي خطأ الطبقة المخفية على  $W.v1$ ؟
5. لماذا نطرح التدرج؟
6. ماذا يحدث إذا كان  $\eta$  كبيراً جداً؟
7. ما وظيفة  $\epsilon$ ؟

# References | المراجع

المراجع العامة المستخدمة في هذا المقرر

[1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.

[3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.

[4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

# Publication Information | معلومات النشر

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| Document         | Laboratory 01 · Neural Network Training                       |
| Course           | AI Applications in Electrical Power Systems                   |
| Author           | Dr.-Ing. Aouss Gabash                                         |
| Version          | 1.0                                                           |
| Publication year | 2026                                                          |
| Official website | <a href="https://aoussgabash.com">https://aoussgabash.com</a> |
| Citation style   | IEEE                                                          |

**Recommended citation:**

Gabash, A. (2026). Neural Network Training. AI Applications in Electrical Power Systems, Laboratory 01, Version 1.0. <https://aoussgabash.com>

---

المرجع المقترح:

أوس غباش، 2026، مخبر 01، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

---

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.